

Exponential-Range Mass Production of Boolean Functions

Local disjoint recovery and recursive congestion reduction

Samuel Schlesinger

September 2026

Abstract

How many independent inputs can one circuit evaluate for essentially the worst-case cost of evaluating a Boolean function once? For $f : \{0, 1\}^n \rightarrow \{0, 1\}$, let $f^{\times t}$ evaluate f on t independent inputs. Uhlig showed that $t = 2^{o(n/\log n)}$ copies preserve the sharp one-copy asymptotic. We prove that, if only the optimal order of growth is required, the same-cost plateau extends to every fixed exponential rate below the 2^n -copy scale: for every fixed $0 \leq \gamma < 1$,

$$C(f^{\times t}) = O_\gamma(2^n/n) \quad \text{for all } t \leq 2^{\gamma n}.$$

The bound is uniform over all f and has the optimal worst-case order, but it does not retain Uhlig's one-copy leading constant.

The proof replaces a global truth-table scan by local coded recovery. It encodes restrictions of f with a constant-rate multivariate evaluation code, recovers requested symbols along affine lines, and schedules the lines to be disjoint inside request groups. The remaining inter-group congestion is bounded and is absorbed by a recursive call on fewer input variables. At the level of admissible copy exponents, the recursion gives

$$\beta \mapsto \frac{1}{2 - \beta}.$$

Starting from the disjoint range $\beta = 1/2$, iteration reaches every constant below one.

AI systems contributed substantively to the development of this work. The project originated in a conversation on ChatGPT.com that was exported to the author's computer as a $\text{\LaTeX}$ document. Later interactive sessions used Claude Fable 5 and 5.1 through Claude Code and ChatGPT 5.6 Sol through Codex for experimentation, broad scans of the literature, and the development of multiple core components of the proof strategy and its internal constructions. The author is responsible for the final arguments and citations.

1 Introduction

How much circuit size can be saved when the same function must be evaluated on many unrelated inputs? This is a direct-sum question in which the outputs are independent but the circuit implementing them need not be: intermediate computations may be reorganized and shared across copies.

For a Boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ and an integer $t \geq 1$, define

$$f^{\times t}(x^{(1)}, \dots, x^{(t)}) = (f(x^{(1)}), \dots, f(x^{(t)})).$$

The naive construction uses t disjoint copies of a circuit for f and gives

$$C(f^{\times t}) \leq tC(f).$$

The mass-production problem asks how large t can become before the cost must rise above the worst-case scale for one copy. For unrestricted circuits, the naive direct-sum bound can be far from optimal. Uhlig's theorem allows $t = 2^{o(n/\log n)}$ independent evaluations with no asymptotic increase in the leading worst-case one-copy synthesis cost; the numerical leading constant depends on the circuit basis and cost convention. See Uhlig's later account [12], Wegener's Theorem 10.2.3 [13], and the modern restatement in [6]; the 1974 paper [11] is a precursor.

We prove, by a different construction, that the order-of-growth plateau reaches $2^{\gamma n}$ simultaneous evaluations for every fixed $\gamma < 1$. The price is that we control only the optimal worst-case order $2^n/n$, not the sharp one-copy leading constant.

1.1 The result and how to read it

We fix the standard basis $\{\wedge, \vee, \neg\}$, with fan-in two for the binary gates. Fan-out and output designations are free, and circuit size is the number of non-input gates. Any other fixed finite complete basis changes the result by at most a constant-factor simulation.

Theorem 1.1 (Exponential-range mass production). *For every fixed $0 \leq \gamma < 1$, there is a constant A_γ such that, for every integer $n \geq 1$, every Boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}$, and every integer $1 \leq t \leq 2^{\gamma n}$,*

$$C(f^{\times t}) \leq A_\gamma \frac{2^n}{n}.$$

Three features of the statement are worth separating. First, it is uniform: one constant A_γ works for every function and for every batch size in the stated range. Second, the upper bound has no factor depending on t . Thus, for $t = \lfloor 2^{\gamma n} \rfloor$, the amortized size per requested value is

$$\frac{C(f^{\times t})}{t} = O_\gamma \left(\frac{2^{(1-\gamma)n}}{n} \right).$$

Third, γ is fixed before n grows. Neither the constant nor the proof is uniform as $\gamma \rightarrow 1$.

The following elementary bound does not explain the fixed-exponent gap, but it shows that the plateau cannot extend to the 2^n -copy scale. If $n \geq 2$ and f depends on all n variables, then every circuit for $f^{\times t}$ has size at least $tn/2$. Consequently, a uniform $O(2^n/n)$ plateau cannot persist once $t = \omega(2^n/n^2)$; see Section 8. Thus Theorem 1.1 reaches every fixed exponential rate while leaving a genuine near-endpoint transition to be understood.

1.2 Global lookup versus local recovery

Paul's general many-copy construction provides a useful contrast [9, Lemma 2]. It treats the truth table of f as an ordered database: sort the t input addresses, sweep once through the hardwired table, and restore the answers to their original order. This global lookup strategy works for arbitrary t and gives

$$O\left(\max\{n2^n, nt \log^2 t\}\right).$$

For every fixed $\gamma < 1$ and $t \leq 2^{\gamma n}$, the full-table term $n2^n$ eventually dominates.

Our construction does not sweep through the full table. It turns the table into a family of shorter resource functions and reconstructs each requested value from a local, selectable set of those resources. The central issue is then not lookup but *simultaneous demand*: how many distinct suffixes ask for the same resource at once? Local disjointness and recursion keep that demand bounded.

	Paul's global lookup	This paper's local recovery
Representation	the complete truth table	coded restrictions indexed by shorter resource functions
Serving a batch	sort addresses, scan the table, and unsort answers	choose one affine-line recovery set per request
Controlling overlap	all requests share the same global scan	disjointness within groups and at most C uses per resource across groups
Resulting tradeoff	arbitrary t , but $O(n2^n)$ in the fixed exponential range	$t \leq 2^{\gamma n}$, but the optimal order $O_\gamma(2^n/n)$

The two methods therefore share sorting as a circuit primitive but use it for different purposes. Paul sorts the input addresses to enable a global table scan. We sort scheduling and routing records after the local recovery sets have been selected.

1.3 The proof in four moves

The construction can be read as four successive transformations.

- 1. Pack restrictions.** Split each input as $x = (a, z)$. For every fixed suffix z , pack the values $f(a, z)$ over all prefixes a into the information symbols of a constant-rate low-degree evaluation code over $\mathbb{F}_q$.
- 2. Recover locally.** A requested information symbol can be reconstructed from the $q - 1$ non-target points on any affine line through it. About $q^{\ell-1}$ directions are available, so every request has many alternative recovery sets.
- 3. Schedule bounded congestion.** Partition the requests into C groups. A deterministic greedy circuit chooses pairwise disjoint recovery lines inside each group. Consequently, each code coordinate is requested on at most one suffix per group and at most C suffixes overall.
- 4. Recurse on the suffix.** For a fixed code coordinate, evaluating its resource function on those at most C suffixes is itself a C -copy problem, now on only $d = |z|$ variables. The recursive hypothesis pays for this remaining simultaneous demand.

Sections 3 to 6 implement these moves and account for their gate costs. Section 7 chooses the parameters and iterates the resulting exponent gain.

1.4 The exponent bootstrap, in outline

Here is the parameter logic behind the recursion. To handle $t \leq 2^{\gamma n}$ requests, the suffix has length $d \approx \gamma n$ and the number C of request groups has exponent c , so a resource sees at most 2^{cn} suffixes. A previously admissible exponent β handles that recursive demand when $c < \beta\gamma$, while the scheduling

overhead requires $c > 2\gamma - 1$, up to slack that vanishes as the fixed number ℓ of code variables grows. Thus a feasible c exists precisely when $\gamma < 1/(2 - \beta)$.

Writing $\text{Adm}(\beta)$ for the admissible-exponent property defined formally in [Definition 7.1](#), a globally disjoint schedule gives $\text{Adm}(1/2)$, and the grouped construction proves

$$\text{Adm}(\beta) \implies \text{Adm}\left(\frac{1}{2 - \beta}\right).$$

Starting from $\beta_1 = 1/2$, the resulting sequence is

$$\beta_k = \frac{k}{k + 1},$$

which converges to one.

1.5 Relation to earlier work

The relevant verified guarantees are summarized below. Here “one-copy leading term” refers to the basis-dependent Shannon–Lupanov worst-case asymptotic.

Result	Copy range	Circuit-size bound	Comparison
Uhlig [12, 13, 6]	$\log t = o(n/\log n)$	one-copy leading term	optimal order and leading constant in a smaller copy range
Paul, Lemma 2 [9]	arbitrary t	$O(\max\{n2^n, nt \log^2 t\})$	$O(n2^n)$ throughout every fixed exponential range below one
Albrecht, Theorem 1 [1]	$t \leq 2^n$	$O(2^n n^2)$	the full exponential range, with polynomial overhead
This paper	$t \leq 2^{\gamma n}$ for fixed $\gamma < 1$	$O_\gamma(2^n/n)$	optimal worst-case order, without leading-constant control

For fixed $\gamma < 1$ and $t \leq 2^{\gamma n}$, Paul’s second term is $o(n2^n)$, so his displayed bound is $O(n2^n)$. The theorem proved here improves that bound by a factor of n^2 at the level of polynomial prefactors. Paul’s g^k is the same k -fold product task considered here; his circuits permit arbitrary binary Boolean gates, which changes our fixed basis cost by at most a constant factor.

Albrecht studies the same simultaneous-realization problem over a complete binary basis [\[1, Theorem 1\]](#). His construction handles every $t \leq 2^n$ with size $O(2^n n^2)$ and depth $O(n \log n)$. Since the one-copy Shannon size is $\Theta(2^n/n)$, this is an $O(n^3)$ multiplicative overhead, as Albrecht also notes. Thus his theorem covers the endpoint copy range but does not give a constant-factor plateau at the one-copy scale.

The coding interpretation begins with Uhlig’s two-copy construction, which is the length- $(2^p + 1)$ single-parity-check code viewed as a two-request disjoint recovery code. Systematic multivariate polynomial evaluation, recovery on affine lines, and the use of disjoint recovery sets appear in the batch-code literature, especially Section 3.4 of Ishai–Kushilevitz–Ostrovsky–Sahai [\[5\]](#). Their geometric procedure chooses lines randomly and deletes intersections. The construction here instead gives a deterministic greedy selection circuit, partitions requests into groups, recursively absorbs the resulting inter-group congestion, and charges the scheduler and fixed-wire routing explicitly. We make no priority claim for these refinements.

A 2011 manuscript of Lipmaa, subsequently withdrawn, proposed computational batch codes in connection with multi-query computation [7]. Later lifted-polynomial constructions also use line-based recovery to obtain batch-code guarantees [4]. PIR codes of Fazeli, Vardy, and Yaakobi form a related family in which each information symbol has many disjoint recovery options [3]. We use only the elementary product-code and affine-line facts proved below.

Independently of historical priority, the technical contribution established here is a self-contained deterministic disjoint-line scheduler, an explicit fixed-wire scatter/gather implementation, and a recursive bounded-congestion invariant whose exponent map is $\beta \mapsto 1/(2 - \beta)$.

1.6 AI-assisted development and provenance

The original idea for this project emerged in an interactive conversation on ChatGPT.com. That conversation was exported to the author’s computer as a $\text{\LaTeX}$ document and became the local starting point for the paper. Further development took place across a series of interactive sessions using Claude Fable 5 and 5.1 through the Claude Code coding harness and ChatGPT 5.6 Sol through the Codex coding harness. These systems were used for mathematical experimentation, broad scans of the literature, and the development of multiple core aspects of the proof strategy and its internal constructions. Their role was substantive and extended beyond prose editing. The author selected among proposed directions, checked and revised the resulting arguments and literature claims, and takes responsibility for the paper’s claims, proofs, and citations.

2 Circuit conventions and worst-case notation

Our circuits are finite directed acyclic graphs. Source vertices are input bits or the constants zero and one. Internal vertices are labelled by $\wedge$, $\vee$, or $\neg$, with the usual fan-in. A circuit for a map $F : \{0, 1\}^N \rightarrow \{0, 1\}^m$ has m designated output wires, which may be input wires or gate outputs and may repeat. Fan-out, constants, and output designations carry no cost; size is the number of internal gates. We write $C(F)$ for the minimum size of such a circuit. All leading-constant comparisons in this paper refer to this fixed model.

Let

$$L(n) = \max_{f: \{0,1\}^n \rightarrow \{0,1\}} C(f).$$

The Shannon counting lower bound and Lupanov’s synthesis upper bound imply

$$L(n) = \Theta(2^n/n)$$

for every fixed finite complete basis; see [10, 8, 13]. We use only the upper bound.

For $r \geq 1$, define

$$L_r(n) = \max_{f: \{0,1\}^n \rightarrow \{0,1\}} C(f^{\times r}).$$

For every $r \geq 1$, one has $L_r(n) \geq L(n)$: fix all but one of the r input blocks and retain the corresponding output. Thus $2^n/n$ is the optimal worst-case order even when several outputs are requested.

For each f , choose a circuit for $f^{\times r}$ of size at most $L_r(n)$. If a construction supplies fewer than r live inputs, the unused inputs of this circuit may be fixed arbitrarily and their outputs ignored. Thus a bound for $L_r(n)$ also applies to every smaller number of copies.

All circuits below are nonuniform. Field representations, irreducible polynomials, sorting networks, and other finite combinatorial objects may be fixed as part of the circuit for each input length.

We use $\tilde{O}_\ell(X)$ for

$$X \cdot \text{poly}_\ell(n, \log q, \log(t+2), \log(C+2)).$$

Only polynomial factors are suppressed; in the parameter regimes used below, they never affect an exponential rate.

For a positive integer k , write $[k]_0 = \{0, \dots, k-1\}$.

We repeatedly use two standard circuit primitives. First, after representing $\mathbb{F}_{2^b}$ in a polynomial basis, field addition, multiplication, inversion of a nonzero element, and comparison of canonical b -bit representations have circuits of size $\text{poly}(b)$. For example, inversion can be implemented by binary exponentiation to the power $2^b - 2$. Second, R records of width w can be sorted by a fixed sorting network using $O(R \log^2 R)$ compare-exchange units, each of size $O(w)$; see [2]. Sorting by a leading type bit, with original positions used as tie breakers when needed, also gives stable fixed-wire compaction within the same asymptotic bound.

3 The two-copy construction as disjoint recovery

Before introducing the higher-dimensional code, it is helpful to isolate the combinatorial idea already present in Uhlig's two-copy argument. The point of this warm-up is not its numerical bound; it is the invariant that makes sharing possible: choose a representation for each requested value so that the underlying resources are not simultaneously reused.

Fix a prefix length p , put $M = 2^p$, and define the restrictions

$$f_i(z) = f(i, z), \quad i \in \{0, \dots, M-1\}.$$

Uhlig's two-copy construction introduces the following functions; see the proof of Wegener's Theorem 10.2.3 [13] and the overview in [6, Section 1.2]:

$$g_0 = f_0, \quad g_i = f_{i-1} \oplus f_i \quad (1 \leq i < M), \quad g_M = f_{M-1}.$$

Every restriction has two representations:

$$f_i = \bigoplus_{j=0}^i g_j = \bigoplus_{j=i+1}^M g_j.$$

Given requests $i \leq j$, recover f_i from the prefix set $\{0, \dots, i\}$ and f_j from the suffix set $\{j+1, \dots, M\}$. The sets are disjoint, so each resource function g_j is evaluated on at most one suffix.

For two requests arriving in the opposite order, a comparator first orders their prefixes, conditional swaps route the two suffixes to the prefix and suffix recovery branches, and a final swap restores the output order. This costs only $\text{poly}(n)$ gates.

The map $(f_0, \dots, f_{M-1}) \mapsto (g_0, \dots, g_M)$ is, up to an invertible change of information coordinates, the length- $(M+1)$ single-parity-check code. The two representations above are disjoint recovery sets for each information coordinate. This suggests replacing the one-dimensional telescoping code by a constant-rate code with many geometric recovery sets.

Two facts survive into the full construction. The resource functions g_j may themselves be arbitrary; only their incidence with the recovery sets matters. And disjointness is needed only among requests served together. The next sections enlarge the menu of recovery sets and then relax global disjointness to bounded demand.

4 A constant-rate evaluation code

This section builds the recovery system used by the circuit. There are two levels to keep separate. For a fixed suffix z , an evaluation code organizes all values $f(a, z)$ across prefixes. Fixing one code coordinate y across all suffixes then defines a shorter function $G_y(z)$. Coding operates in the prefix direction; recursion operates in the suffix direction.

Fix an integer $\ell \geq 2$ and a field $\mathbb{F}_q$ of characteristic two, where $q = 2^b$. Set

$$s_0 = \left\lfloor \frac{q-1}{\ell} \right\rfloor$$

and fix a polynomial-basis representation of $\mathbb{F}_q$. If α_i denotes the field element whose canonical b -bit representation is the integer i , choose the efficiently indexed set

$$A = \{\alpha_0, \dots, \alpha_{s_0-1}\}.$$

Let $\mathcal{V}$ be the space of polynomials

$$P(X_1, \dots, X_\ell) \in \mathbb{F}_q[X_1, \dots, X_\ell]$$

having degree less than s_0 in every variable. Evaluation on A^ℓ uniquely determines P , and

$$\deg P \leq \ell(s_0 - 1) \leq q - 2.$$

The evaluation map

$$\text{Enc} : \mathbb{F}_q^{A^\ell} \longrightarrow \mathbb{F}_q^\ell, \quad (P(u))_{u \in A^\ell} \longmapsto (P(y))_{y \in \mathbb{F}_q^\ell},$$

is a systematic linear code of length

$$N = q^\ell$$

and dimension

$$K = s_0^\ell = \Theta_\ell(q^\ell).$$

4.1 Packing Boolean restrictions

Split an n -bit input as

$$x = (a, z), \quad |a| = p, \quad |z| = d = n - p.$$

For each fixed suffix z , pack the 2^p bits

$$(f(a, z))_{a \in \{0,1\}^p}$$

into K field symbols, using the fixed $\mathbb{F}_2$ -basis of $\mathbb{F}_q$ and padding unused bit positions with zeros. More explicitly, interpret a prefix a as an integer $I(a) \in [2^p]_0$, and set

$$s(a) = \left\lfloor \frac{I(a)}{b} \right\rfloor, \quad j(a) = I(a) \bmod b.$$

Write $s(a)$ in base s_0 using exactly ℓ digits $i_1(a), \dots, i_\ell(a) \in [s_0]_0$, and define

$$u(a) = (\alpha_{i_1(a)}, \dots, \alpha_{i_\ell(a)}) \in A^\ell.$$

Thus bit $j(a)$ of the information symbol at $u(a)$ is $f(a, z)$. Fixed-divisor division, remainder, and base conversion have Boolean circuits of size $\text{poly}_\ell(p, \log q)$, so computing $(u(a), j(a))$ for all requests costs $\tilde{O}_\ell(t)$ gates.

Choose q , among powers of two, minimally so that

$$Kb \geq 2^p.$$

For all sufficiently large p (depending on ℓ), this also ensures $q > 2\ell + 2$, and minimality together with $K = \Theta_\ell(q^\ell)$ gives

$$q^\ell b = \Theta_\ell(2^p). \quad (1)$$

Indeed, the lower bound follows from $Kb \leq q^\ell b$. For the upper bound, the preceding candidate $q/2$ has bit width $b - 1$, fails the packing inequality, and has dimension $\Omega_\ell(q^\ell)$. Hence $q^\ell(b - 1) = O_\ell(2^p)$; since $b \leq 2(b - 1)$ for $b \geq 2$, the upper bound in (1) follows. Equivalently,

$$\ell \log_2 q + \log_2 \log_2 q = p + O_\ell(1), \quad \log_2 q = \frac{p}{\ell} + O_\ell(\log p). \quad (2)$$

Let P_z be the polynomial whose information symbols are the packed bits. For each code coordinate $y \in \mathbb{F}_q^\ell$, define the field-valued resource

$$G_y(z) = P_z(y).$$

Each of the b coordinate bits of G_y is a Boolean function on d variables, determined by f , y , and the chosen field basis.

It is useful to picture the values $P_z(y)$ as a rectangular array. A row, with z fixed and y varying, is a codeword that contains all requested prefix values. A column, with y fixed and z varying, is the resource function G_y on d variables. The line decoder reads selected entries from one row, while the recursive circuit evaluates the required columns.

4.2 Affine-line recovery

Lemma 4.1 (Line identity). *Let $P : \mathbb{F}_q^\ell \rightarrow \mathbb{F}_q$ be represented by a polynomial of total degree at most $q - 2$. For every $u \in \mathbb{F}_q^\ell$ and every nonzero direction $v \in \mathbb{F}_q^\ell$,*

$$P(u) = - \sum_{\lambda \in \mathbb{F}_q^*} P(u + \lambda v).$$

In characteristic two, the minus sign may be omitted.

Proof. The restriction $Q(\lambda) = P(u + \lambda v)$ is a univariate polynomial of degree at most $q - 2$. The sum of the constant monomial over $\mathbb{F}_q$ is $q \cdot 1 = 0$ in characteristic two. For $1 \leq j \leq q - 2$, the sum of λ^j over $\mathbb{F}_q^*$ vanishes because j is not divisible by $q - 1$, and the zero term contributes nothing. Applying these identities monomial by monomial gives $\sum_{\lambda \in \mathbb{F}_q} Q(\lambda) = 0$. Separating the term $\lambda = 0$ proves the claim. $\square$

Thus every affine line through an information point u supplies a recovery set consisting of its $q - 1$ non-target points. The number of projective directions through a point is

$$D_q = \frac{q^\ell - 1}{q - 1} = 1 + q + \dots + q^{\ell-1}, \quad q^{\ell-1} \leq D_q < 2q^{\ell-1}. \quad (3)$$

For fixed ℓ and $p \rightarrow \infty$, Equations (2) and (3) give the parameter dictionary

$$q^\ell = 2^{p+o(p)}, \quad q = 2^{p/\ell+o(p)}, \quad D_q = q^{\ell-1+o(1)} = 2^{((\ell-1)/\ell)p+o(p)}. \quad (4)$$

Two distinct lines through the same target intersect only at that target, which is absent from both recovery sets. Repeated requests for one information symbol therefore cause no additional difficulty.

In summary, one desired bit $f(a, z)$ has been converted into $q - 1$ requests of the form $G_y(z)$, but the circuit may choose those coordinates y from any of D_q recovery lines. The rest of the construction uses this freedom to keep different requests from overloading the same resource.

5 Deterministic scheduling and routing

5.1 Disjoint scheduling inside one group

There are two obstacles to turning the geometric existence argument into a circuit. Lines chosen for different targets may intersect, and the choice of a nonintersecting line depends on the target points. The greedy argument below handles the first obstacle, while projective ranking and sorting implement the choice with ordinary Boolean gates.

Consider a multiset of g requested information points $u_1, \dots, u_g \in A^\ell$. Process the requests in order. Suppose disjoint recovery sets have been selected for requests $1, \dots, j - 1$, and let U be their union. Then

$$|U| = (j - 1)(q - 1) < gq.$$

For the target u_j , every point $y \in U \setminus \{u_j\}$ forbids at most one projective direction, namely the direction of $y - u_j$. The point u_j itself forbids none because it is omitted from the new recovery set.

Lemma 5.1 (Projective indexing). *Projective directions in $\mathbb{F}_q^\ell$ have rank and unrank circuits of size $\text{poly}_\ell(b)$, with ranks in $[D_q]_0$.*

Proof. Normalize a nonzero vector so that its first nonzero coordinate is one. The normalized directions whose first nonzero coordinate is in position $h \in \{1, \dots, \ell\}$ have the form

$$(0, \dots, 0, 1, w_{h+1}, \dots, w_\ell)$$

and form a block of size $q^{\ell-h}$. Let

$$B_h = \sum_{r=1}^{h-1} q^{\ell-r}, \quad \text{val}_q(w_{h+1}, \dots, w_\ell) = \sum_{r=h+1}^{\ell} \text{int}(w_r) q^{\ell-r},$$

where $\text{int}(w_r) \in [q]_0$ is the canonical binary representation. The rank is

$$B_h + \text{val}_q(w_{h+1}, \dots, w_\ell).$$

The blocks are consecutive and their total size is D_q .

To rank, find the first nonzero coordinate, invert it, normalize the vector, and concatenate the b -bit tail coordinates. To unrank, compare the rank with the ℓ hardwired block intervals, subtract the appropriate B_h , and split the remainder into base- q digits. Since $q = 2^b$, the last step is bit slicing. The field operations and comparisons from the circuit conventions give size $\text{poly}_\ell(b)$ in both directions. $\square$

Lemma 5.2 (Greedy line scheduler). *If*

$$gq < D_q, \tag{5}$$

then every multiset of g target points admits pairwise disjoint affine-line recovery sets. Given the targets, the recovery sets can be selected by a Boolean circuit of size

$$\tilde{O}_\ell(g^2 q).$$

Proof. At step j , fewer than D_q directions are forbidden, so a valid direction exists. This proves the combinatorial statement.

For the circuit bound, represent a projective direction by normalizing its first nonzero coordinate to one and use the ranking from [Lemma 5.1](#). At stage j , for each $y \in U$, output the sentinel rank D_q when $y = u_j$ and otherwise output the projective rank of $y - u_j$. At the gate level, first multiplex a fixed nonzero vector in place of $y - u_j$ when $y = u_j$, rank that always-nonzero vector, and then overwrite the result by D_q on the equality branch. Use $\lceil \log_2(D_q + 1) \rceil$ bits per rank. There are $(j - 1)(q - 1) = O(jq)$ such entries.

Sort these ranks as $r_1 \leq \dots \leq r_m$. The least missing rank can be found without a dense D_q -bit table. Rank zero is a candidate if $m = 0$ or $r_1 > 0$; after position i , rank $r_i + 1$ is a candidate if

$$r_i < D_q - 1 \quad \text{and} \quad (i = m \text{ or } r_{i+1} > r_i + 1).$$

These tests are applied to the full sorted list, including duplicates and sentinels. Only the last occurrence of a non-sentinel rank can generate its successor, and a sentinel cannot generate a candidate. A priority circuit selects the first candidate in this order. A left-to-right prefix OR and multiplexers implement the choice in $O(m \log D_q)$ gates. Since $|U \setminus \{u_j\}| < D_q$, fewer than D_q distinct non-sentinel ranks are forbidden, so a candidate exists. Sorting, comparisons, and priority selection therefore cost $\tilde{O}_\ell(jq)$ gates. Unrank the selected direction using [Lemma 5.1](#) and enumerate its $q - 1$ non-target line points, at cost $q \text{poly}_\ell(b)$. Unrolling the stages and summing over $j \leq g$ gives $\tilde{O}_\ell(g^2 q)$. $\square$

5.2 Request groups

Partition the t requests into C fixed groups, each of size at most

$$g = \left\lceil \frac{t}{C} \right\rceil.$$

Apply Lemma 5.2 independently inside each group. A code coordinate is used at most once per group and therefore at most C times overall. The total scheduling cost is

$$C \cdot \tilde{O}_\ell(g^2 q) = \tilde{O}_\ell\left(\frac{t^2 q}{C} + Cq\right), \quad (6)$$

because

$$C \left\lceil \frac{t}{C} \right\rceil^2 q \leq \frac{t^2 q}{C} + 2tq + Cq \leq 2 \left(\frac{t^2 q}{C} + Cq \right).$$

The last inequality is the arithmetic–geometric mean inequality. The construction is valid whenever (5) holds for $g = \lceil t/C \rceil$, that is, whenever

$$\left\lceil \frac{t}{C} \right\rceil q < D_q. \quad (7)$$

The parameter C exposes the central tradeoff. More groups mean fewer requests per group, making disjoint scheduling easier and cheaper. But they also permit each resource coordinate to receive more suffixes, making the recursive C -copy problem harder. The bootstrap comes entirely from finding an exponential choice of C that satisfies both demands.

5.3 Scatter and gather

Every non-target point of a selected line produces an incidence record containing a request identifier, a group, a line position, the point’s code coordinate y , and the suffix to be evaluated. There are exactly $t(q-1) < tq$ such records. Inside one group, each coordinate appears in at most one record.

The routing consists of four fixed sorting-and-local-update passes. Here “key” means (group, y) ; any unlisted tie is broken by the record’s original position.

Pass	Records	Sort order	Local result
Scatter match	incidence, slot	(key, type); incidence before slot	a matched slot takes its predecessor’s suffix; otherwise it takes a dummy
Scatter order	incidence, filled slot	(type, key); slots first	slots are in canonical (group, y) order
Gather match	value, saved incidence	(key, type); value before incidence	a matched incidence takes its predecessor’s value
Gather order	value, updated incidence	(type, request, line position); incidences first	decoder inputs are in canonical request order

Scatter records carry a d -bit suffix. After resource evaluation, value records carry the b -bit field value $G_y(z)$. The gather passes are payload-agnostic for any payload whose width is polynomial in the displayed parameters.

Lemma 5.3 (Record routing). *The suffixes can be scattered into slots indexed by $(\text{group}, y) \in [C]_0 \times \mathbb{F}_q^\ell$, and the resulting resource values can be gathered back to their request records, using circuits of total size*

$$\tilde{O}_\ell(tq + Cq^\ell).$$

Proof. For scatter, create one slot record for each of the Cq^ℓ keys and combine them with the fewer than tq incidence records. Preserve a copy of every incidence record by free fan-out. Uniqueness inside a group puts at most one incidence at each key. After the first pass in the table, a slot therefore copies the preceding suffix exactly when a type-and-key equality test succeeds; otherwise it receives a fixed dummy suffix. The predecessor of the first position is defined to be a dummy record, so the boundary case uses the same guard. The second pass places the filled slots in canonical (group, y) order. Fixed wiring transposes this order to (y, group) , the input order used by the resource circuits.

For gather, combine the preserved incidences with one value record per key. The third pass places that value immediately before every matching incidence; a guarded local multiplexer copies it into the incidence. The fourth pass puts the first $t(q-1)$ positions in the fixed request-and-line order required by the decoders. Thus absent incidences, the first array position, and all type boundaries are handled by fixed dummy data and the same guarded update, not by data-dependent wiring.

This is a constant number of sorting networks on $O(tq + Cq^\ell)$ records. Each record has width $\text{poly}_\ell(n, \log q, \log t, \log C)$, so the standard primitives from the circuit conventions give the stated bound. $\square$

The lemma is the reason no tq^ℓ crossbar appears. Only the actual incidences and one dummy record per resource slot are routed.

6 The master recurrence

Bounded-demand invariant. Fix f and the packing above, and for $y \in \mathbb{F}_q^\ell$ and $k \in [b]_0$ let

$$H_{y,k}(z) = (G_y(z))_k.$$

This is one fixed Boolean function on d variables. Inside each request group, disjoint recovery ensures that coordinate y occurs at most once; hence the pair (y, k) receives at most one suffix from each of the C groups. Place those suffixes in the canonical group slots and pad unused slots with a fixed dummy suffix. One circuit for $H_{y,k}^{\times C}$, of size at most $L_C(d)$, serves exactly this bounded demand. There are $q^\ell b$ such pairs, and no sharing between different pairs is assumed.

Proposition 6.1 (Master bound). *Fix $\ell \geq 2$. Let $n = p + d$ with p sufficiently large depending on ℓ , choose $q = 2^b$ minimally so that $s_0^\ell b \geq 2^p$, and suppose Equation (7) holds. Then every $f : \{0, 1\}^n \rightarrow \{0, 1\}$ satisfies*

$$C(f^{\times t}) \leq \underbrace{q^\ell b L_C(d)}_{\text{recursive resource evaluation}} + \tilde{O}_\ell \left(\underbrace{\frac{t^2 q}{C}}_{\text{line scheduling}} + \underbrace{tq}_{\text{incidences and decoding}} + \underbrace{Cq^\ell}_{\text{resource slots}} \right). \quad (8)$$

The first term is the only recursive term: there are $q^\ell b$ Boolean resource functions, and each is evaluated on at most C suffixes of length d . The other three terms are the complete bookkeeping bill. Increasing C decreases the scheduling term but increases both the recursive demand and the number of resource slots. The bootstrap will choose C so that the first term stays at scale $2^n/n$ while every bookkeeping term is exponentially smaller.

Proof. Map every requested prefix a to its information point $u(a)$ and bit position $j(a)$ using the canonical packing circuit. In each request group, use [Lemma 5.2](#) to select and enumerate a recovery line. Together these steps cost $\tilde{O}_\ell(t) + \tilde{O}_\ell(t^2q/C + Cq)$ gates. Each non-target point y on a selected line requests G_y on the suffix of the corresponding output instance.

For each pair (y, k) , use the $H_{y,k}^{\times C}$ circuit from the bounded-demand invariant on the canonical group slots. The definition of $L_C(d)$ applies separately to each induced function $H_{y,k}$, so these circuits cost $q^\ell b L_C(d)$ in total. The complete scatter, gather, and fixed-wire compaction pipeline costs $\tilde{O}_\ell(tq + Cq^\ell)$ by [Lemma 5.3](#). Field addition is coordinatewise XOR in the fixed $\mathbb{F}_2$ basis, so XORing the $q - 1$ returned b -bit values of a request and selecting bit $j(a)$ costs $O(qb)$ per request and $O(tqb)$ in total. Since each selected line passes through its target $u(a)$ and $\deg P_z \leq q - 2$, [Lemma 4.1](#) shows that this XOR equals $P_z(u(a))$ for the request's suffix z , and bit $j(a)$ of that symbol is $f(a, z)$ by the packing of [Section 4.1](#). The circuit therefore computes $f^{\times t}$.

The polynomial P_z and the functions G_y specify the nonuniform resource truth tables; no run-time encoder is being assumed. Since $q \geq 2$, all smaller terms in this gate ledger are absorbed by the overhead in (8), proving the bound. $\square$

7 The exponent bootstrap

The master bound becomes transparent at the level of exponential rates. Suppose batches of size $2^{\beta m}$ are already known to cost $O(2^m/m)$ on m variables. To handle $t = 2^\gamma$ copies, take a suffix of length $d \approx \gamma n$ and $C \approx 2^{cn}$ request groups. Ignoring the slack from the fixed code dimension ℓ , the two competing requirements are

$$\underbrace{2\gamma - 1}_{\text{scheduling and routing overhead}} < c < \underbrace{\beta\gamma}_{\text{recursive capacity}}. \quad (9)$$

The left inequality makes the nonrecursive terms sub- 2^n ; the right lets the known β -range handle C copies on d variables. This window is nonempty exactly when $\gamma < 1/(2 - \beta)$. Everything below makes this calculation uniform, restores the finite- ℓ slack, and accounts for floors, ceilings, and polynomial factors.

Definition 7.1 (Admissible exponent property). For $0 < \beta \leq 1$, let $\text{Adm}(\beta)$ denote the following assertion. For every fixed $0 < \rho < \beta$, there is a constant $A_{\beta,\rho}$ such that, for all sufficiently large m , every $h : \{0, 1\}^m \rightarrow \{0, 1\}$, and every integer $1 \leq r \leq 2^{\rho m}$,

$$C(h^{\times r}) \leq A_{\beta,\rho} \frac{2^m}{m}.$$

The constant and the threshold may depend on the fixed exponents β and ρ , but not on m , h , or r .

Lemma 7.2 (Strict exponential margins). Suppose $X_n, Y_n > 0$ and, for fixed $a < b$,

$$\log_2 X_n \leq an + o(n), \quad \log_2 Y_n \geq bn + o(n).$$

Then $R(n)X_n < Y_n$ for every fixed polynomial R and all sufficiently large n . Also, for every fixed $a \geq 0$ and $\eta > 0$, $\lceil 2^{an} \rceil \leq 2^{(a+\eta)n}$ for all sufficiently large n .

Proof. Choose a positive margin smaller than $(b - a)/3$. The two $o(n)$ terms and $\log_2 R(n) = o(n)$ are eventually smaller than this margin times n , which proves the first claim. The second follows from $\lceil 2^{an} \rceil \leq 2^{an} + 1 \leq 2^{an+1}$ and $an + 1 \leq (a + \eta)n$ once $n \geq 1/\eta$. $\square$

7.1 The disjoint base range

Lemma 7.3 (Base range). *Adm(1/2) holds.*

Here $C = 1$: all recovery sets are globally disjoint, so no recursive mass-production hypothesis is needed. The one-half threshold comes from the quadratic scheduling cost, not from an intrinsic shortage of directions. If $d = \delta n$, then as ℓ grows the line-availability condition approaches $\gamma < 1 - \delta$, whereas the term $t^2 q$ still forces $\gamma < 1/2$. We take $\delta = \gamma$ below to match the parameterization used in the bootstrap.

Proof. Fix $0 < \gamma < 1/2$ and let $t \leq 2^{\gamma n}$. Choose

$$d = \lfloor \gamma n \rfloor, \quad p = n - d, \quad C = 1,$$

and write $\vartheta = 1 - \gamma$. By [Equation \(4\)](#),

$$\log_2 q = \frac{\vartheta n}{\ell} + o(n), \quad \log_2 q^\ell = \vartheta n + o(n).$$

Choose a sufficiently large fixed ℓ so that

$$\gamma + \frac{\vartheta}{\ell} < \vartheta \left(1 - \frac{1}{\ell}\right). \quad (10)$$

This is possible precisely because $\gamma < 1/2$. Equations (3) and (10), together with [Lemma 7.2](#), imply $tq < D_q$, so the scheduler is valid for all sufficiently large n .

The resource term in [Equation \(8\)](#) is, by Shannon–Lupanov synthesis,

$$q^\ell bL(d) = O_\ell \left(2^p \frac{2^d}{d} \right) = O_\gamma(2^n/n).$$

The scheduler exponent is

$$2\gamma + \frac{\vartheta}{\ell} < 1,$$

where the strict inequality follows from the stronger margin (10). The tq and q^ℓ terms also have exponents below one. After the suppressed polynomial factors are restored, every overhead term is $2^{(1-\Omega_\gamma(1))n}$ and is negligible compared with $2^n/n$ by [Lemma 7.2](#). The same strict margins absorb the floor in d and the field-size rounding. $\square$

7.2 Recursive congestion reduction

Lemma 7.4 (Bootstrap). *If Adm(β) holds for some $0 < \beta \leq 1$, then*

$$\text{Adm}\left(\frac{1}{2-\beta}\right)$$

holds.

Proof. To verify the asserted property, fix a target exponent $0 < \gamma < 1/(2 - \beta)$. The range $\gamma < 1/2$ is already covered by [Lemma 7.3](#). If $\gamma = 1/2$, choose $\hat{\gamma} \in (1/2, 1/(2 - \beta))$; the construction below for $\hat{\gamma}$ is uniform for every $t \leq 2^{\hat{\gamma}n}$ and hence, by monotonicity of the allowed range, also handles every $t \leq 2^{\gamma n}$. We may therefore assume

$$\frac{1}{2} < \gamma < \frac{1}{2 - \beta}.$$

The upper inequality is equivalent to

$$2\gamma - 1 < \beta\gamma.$$

Choose a constant c such that

$$2\gamma - 1 < c < \beta\gamma. \quad (11)$$

Put

$$d = \lfloor \gamma n \rfloor, \quad p = n - d, \quad C = \lceil 2^{cn} \rceil.$$

Choose ρ with

$$\frac{c}{\gamma} < \rho < \beta.$$

Since $c < \rho\gamma$, the bounds $C \leq 2^{cn} + 1 \leq 2^{cn+1}$ and $d \geq \gamma n - 1$ imply $C \leq 2^{\rho d}$ for all sufficiently large n . Thus $\text{Adm}(\beta)$ gives

$$L_C(d) = O_{\beta, \rho}(2^d/d).$$

By [Equation \(1\)](#), the resource term is therefore

$$q^\ell b L_C(d) = O_{\beta, \gamma, \rho, \ell}(2^n/n).$$

Set $\vartheta = 1 - \gamma$. The request-group size has exponent $\gamma - c$. Choose a sufficiently large fixed ℓ so that

$$c > 2\gamma - 1 + \frac{2\vartheta}{\ell}. \quad (12)$$

This is possible by the left inequality in (11). The scheduling condition follows because

$$\gamma - c + \frac{\vartheta}{\ell} < \vartheta \left(1 - \frac{1}{\ell}\right).$$

Indeed, $\lceil t/C \rceil \leq 2^{(\gamma-c)n} + 1$, while [Equations \(3\) and \(4\)](#) and [Lemma 7.2](#) turn this strict exponent margin into [Equation \(7\)](#) for all sufficiently large n .

The three overhead exponents in [Equation \(8\)](#) are bounded by

$$\begin{aligned} \frac{1}{n} \log_2 \left(\frac{t^2 q}{C} \right) &\leq 2\gamma - c + \frac{\vartheta}{\ell} + o(1) < 1, \\ \frac{1}{n} \log_2(tq) &\leq \gamma + \frac{\vartheta}{\ell} + o(1) < 1, \\ \frac{1}{n} \log_2(Cq^\ell) &= c + \vartheta + o(1) < 1. \end{aligned}$$

The first inequality follows from (12), the second from $\ell > 1$, and the third from $c < \beta\gamma \leq \gamma$. Thus every overhead term is exponentially smaller than $2^n/n$ by [Lemma 7.2](#). All choices in this one-step argument are fixed constants. In the final finite iteration they depend only on the target exponent γ , so their constants combine into A_γ . $\square$

The first few iterations are therefore

$$\frac{1}{2} \mapsto \frac{2}{3} \mapsto \frac{3}{4} \mapsto \frac{4}{5} \mapsto \dots$$

Each round allows more congestion in exchange for recursing on the shorter suffix. Any fixed target $\gamma < 1$ is crossed after finitely many rounds.

Proof of Theorem 1.1. For $\gamma = 0$, the Shannon–Lupanov upper bound proves the claim for all sufficiently large n . Now suppose $0 < \gamma < 1$. Starting from $\beta_1 = 1/2$, define

$$\beta_{k+1} = \frac{1}{2 - \beta_k}.$$

An induction gives $\beta_k = k/(k+1)$. By Lemmas 7.3 and 7.4, $\text{Adm}(\beta_k)$ holds for every k . Choose k such that $\gamma < k/(k+1)$ and apply $\text{Adm}(\beta_k)$ with $\rho = \gamma$. In either case, the asserted bound holds beyond a threshold N_γ . For the finitely many integers $1 \leq n < N_\gamma$, use $C(f^{\times t}) \leq tC(f)$ and enlarge A_γ to dominate $ntC(f)/2^n$ over all relevant n, f , and t . This proves the statement for every $n \geq 1$. $\square$

Remark 7.5 (How to place the result). The theorem improves one coordinate of the known tradeoff, not all of them at once. Compared with Uhlig, it exchanges the sharp one-copy leading asymptotic for an exponentially larger copy range. Compared with Paul, it exchanges an arbitrary copy range for the optimal $2^n/n$ worst-case order throughout every fixed exponential range below one. In particular, the theorem is a plateau statement at the level of exponential rates. It neither identifies the sharp leading constant nor locates the transition when the copy exponent approaches one with n .

8 Discussion and limitations

The scheduler bottleneck. The disjoint base case isolates the reason for the bootstrap. With $C = 1$ and $d = \delta n$, the resource term remains $O(2^n/(\delta n))$, and for large fixed ℓ there are enough directions whenever $\gamma < 1 - \delta$. It is the quadratic scheduling term $t^2 q$, not line availability, that restricts this argument to $\gamma < 1/2$. Grouping divides that term by C , while recursion pays for the resulting C simultaneous uses of each resource; the map $\beta \mapsto 1/(2 - \beta)$ records exactly this exchange. A disjoint-line scheduler of size $\tilde{O}(tq)$ would remove the need for recursion: for any fixed $\gamma < 1$, one could choose $0 < \delta < 1 - \gamma$ and take $C = 1$. Finding such a scheduler, or ruling one out, is therefore a natural next problem.

A consequence for lower-bound measures. Suppose a nonnegative measure μ on Boolean vector functions satisfies $\mu(F) \leq C(F)$ and $\mu(f^{\times r}) \geq r\mu(f)$. Taking $r = \lfloor 2^{(1-\varepsilon)n} \rfloor$ for fixed $0 < \varepsilon < 1$ in Theorem 1.1 gives, uniformly in $f : \{0, 1\}^n \rightarrow \{0, 1\}$,

$$\mu(f) = O_\varepsilon(2^{\varepsilon n}/n),$$

and hence a uniformly subexponential worst-case value. Paul’s general bound already gives the qualitative conclusion, with $O_\varepsilon(n2^{\varepsilon n})$ in place of the displayed bound; the new content is the factor- n^2 improvement in the polynomial prefactor. This is an abstract constraint on lower-bound strategies, not an application to a standard named measure: any useful measure satisfying the first inequality must fail to charge independent copies fully or must otherwise represent cross-instance sharing.

Leading constants. The resource term pays for q^ℓ code symbols while carrying s_0^ℓ information symbols. Since s_0/q tends to $1/\ell$ for fixed ℓ as q grows, the ratio q^ℓ/s_0^ℓ is roughly ℓ^ℓ . Moreover, synthesizing the resource functions separately introduces the factor n/d , which is approximately $1/\gamma$ in a bootstrap step. Thus, normalized by $2^n/n$, the recursive coefficient is multiplied by roughly ℓ^ℓ/γ even before field-size rounding, and repeated bootstrapping compounds the loss. The scheduler and routing terms are exponentially lower order for every fixed exponent margin, so improving their constant factors alone would not recover a sharp leading term. Our argument therefore does not determine whether the one-copy Shannon–Lupanov leading constant for this basis can be retained for any fixed exponential number of copies. We make no claim about whether the existing literature settles that question.

The endpoint. The theorem reaches $2^{(1-\varepsilon)n}$ copies for every fixed $\varepsilon > 0$, but does not treat a rate that approaches one with n . Some degradation in that regime is forced. If $n \geq 2$ and f depends on all n of its variables, every output of a circuit for $f^{\times t}$ is a gate, and each of the tn inputs must feed at least one gate, since otherwise the corresponding output would not depend on it. Since gates have fan-in at most two,

$$C(f^{\times t}) \geq \frac{tn}{2}.$$

A bound of the form $A 2^n/n$ therefore cannot hold once $t > 2A \cdot 2^n/n^2$; in particular, a uniform $O(2^n/n)$ plateau must fail for $t = \omega(2^n/n^2)$. The theorem still leaves the detailed mass-production curve throughout the near-endpoint regime $t = 2^{n-o(n)}$ unresolved. Determining how far the plateau extends, and then characterizing the growth beyond it, would require a quantitatively sharper schedule or a different recursive decomposition.

Historical scope. The introduction compares the stated guarantees of Uhlig, Paul, and Albrecht. These theorem-level comparisons do not assert that the coding, scheduling, or routing ingredients are individually new. The self-contained contribution claimed here is their deterministic bounded-congestion combination and the resulting exponent bootstrap.

Restricted models. The construction uses arbitrary routing, finite-field linear reconstruction, and free fan-out. Formula size, bounded fan-out circuits, monotone circuits, bounded-depth circuits, and branching programs may have different mass-production curves. In those settings, simultaneous demand may become a source of genuine direct-sum lower bounds.

Other recovery geometries. Affine lines provide many recovery choices with simple intersection behavior, but their recovery sets have size $q - 1$. Batch codes, locally recoverable codes, or higher-dimensional varieties may improve the group-size versus decoding-cost tradeoff and reduce the number of bootstrap rounds.

References

- [1] Andreas Albrecht. On simultaneous realizations of Boolean functions, with applications. In *Parcella '88: Proceedings of the Fourth International Workshop on Parallel Processing by Cellular Automata and Arrays*, LNCS 342, pages 51–56. Springer, 1989. doi:10.1007/3-540-50647-0_102.
- [2] Kenneth E. Batcher. Sorting networks and their applications. In *Proceedings of the 1968 Spring Joint Computer Conference*, AFIPS Conference Proceedings 32, pages 307–314, 1968. doi:10.1145/1468075.1468121.
- [3] Arman Fazeli, Alexander Vardy, and Eitan Yaakobi. PIR with low storage overhead: Coding instead of replication. arXiv:1505.06241, 2015. <https://arxiv.org/abs/1505.06241>.
- [4] Lukas Holzbaur, Rina Polyanskaya, Nikita Polyanskii, and Ilya Vorobyev. Lifted Reed–Solomon codes with application to batch codes. In *2020 IEEE International Symposium on Information Theory*, pages 634–639. IEEE, 2020. doi:10.1109/ISIT44484.2020.9174402.
- [5] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Batch codes and their applications. In *Proceedings of the 36th Annual ACM Symposium on Theory of Computing*, pages 262–271. ACM, 2004. doi:10.1145/1007352.1007396.
- [6] William Kretschmer. Quantum mass production theorems. In *18th Conference on the Theory of Quantum Computation, Communication and Cryptography*, LIPIcs 266, Article 10, pages 10:1–10:11, 2023. doi:10.4230/LIPIcs.TQC.2023.10.
- [7] Helger Lipmaa. Efficient multi-query CPIR from Ring-LWE. IACR Cryptology ePrint Archive, Report 2011/595, 2011. Withdrawn in 2014. <https://eprint.iacr.org/2011/595>.
- [8] Oleg B. Lupanov. On a method of circuit synthesis. *Izvestiya VUZ, Radiofizika*, 1(1):120–140, 1958. In Russian. <https://radiophysics.unn.ru/issues/1958/1/120>.
- [9] Wolfgang J. Paul. Realizing Boolean functions on disjoint sets of variables. *Theoretical Computer Science*, 2(3):383–396, 1976. doi:10.1016/0304-3975(76)90089-X.
- [10] Claude E. Shannon. The synthesis of two-terminal switching circuits. *Bell System Technical Journal*, 28(1):59–98, 1949. doi:10.1002/j.1538-7305.1949.tb03624.x.
- [11] Dietmar Uhlig. On the synthesis of self-correcting schemes from functional elements with a small number of reliable elements. *Mathematical Notes of the Academy of Sciences of the USSR*, 15(6):558–562, 1974. doi:10.1007/BF01152835.
- [12] Dietmar Uhlig. Networks computing Boolean functions for multiple input values. In M. S. Paterson, editor, *Boolean Function Complexity*, London Mathematical Society Lecture Note Series 169, pages 165–173. Cambridge University Press, 1992. doi:10.1017/CBO9780511526633.013.
- [13] Ingo Wegener. *The Complexity of Boolean Functions*. Wiley–Teubner, 1987. https://eccc.weizmann.ac.il/static/books/The_Complexity_of_Boolean_Functions/.